conformal_glioma_segmentation

October 5, 2026

[1]:
```
# Importing all necessary libraries
import os
import cv2
import glob
import PIL
import shutil
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import plotly.graph_objects as go
from skimage import data
from skimage.util import montage
import skimage.transform as skTrans
from skimage.transform import rotate
from skimage.transform import resize
from PIL import Image, ImageOps


# neural imaging
import nilearn as nl
import nibabel as nib
import nilearn.plotting as nlplt
# !pip install git+https://github.com/miykael/gif_your_nifti # nifti to gif
# import gif_your_nifti.core as gif2nif


# ml libs
import keras
import keras.backend as K
from keras.callbacks import CSVLogger
from keras.layers import Flatten
import tensorflow as tf
from tensorflow.keras.utils import plot_model
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
```

```
from tensorflow.keras.models import *
from tensorflow.keras.layers import *
from tensorflow.keras.optimizers import *
from tensorflow.keras.callbacks import ModelCheckpoint, ReduceLROnPlateau,␣
 ↪EarlyStopping, TensorBoard
#from tensorflow.keras.layers.experimental import preprocessing

# Make numpy printouts easier to read.
np.set_printoptions(precision=3, suppress=True)
```

[2]:
```
# DEFINE seg-areas
SEGMENT_CLASSES = {
    0 : 'NOT tumor',
    1 : 'NECROTIC/CORE', # or NON-ENHANCING tumor CORE
    2 : 'EDEMA',
    3 : 'ENHANCING' # original 4 -> converted into 3 later
}

# there are 155 slices per volume
# to start at 5 and use 145 slices means we will skip the first 5 and last 5
VOLUME_SLICES = 100
VOLUME_START_AT = 22 # first slice of volume that we will include
IMG_SIZE=128

TRAIN_DATASET_PATH = os.path.join(os.getcwd(), 'BraTS2020_TrainingData/
 ↪MICCAI_BraTS2020_TrainingData/')
VALIDATION_DATASET_PATH = os.path.join(os.getcwd(), 'BraTS2020_ValidationData/
 ↪MICCAI_BraTS2020_ValidationData/')
```

1 Neural Network Tumor Segmentation Classifier Starts Here

1.1 Base model from Kaggle see https://www.kaggle.com/code/rastislav/3d-mri-brain-tumor-segmentation-u-net made by Ratislav Kopál

1.2 (Adjusted to required specification)

[3]:
```
# dice loss as defined above for 4 classes
def dice_coef(y_true, y_pred, smooth=1.0):
    class_num = 4
    for i in range(class_num):
        y_true_f = K.flatten(y_true[:,:,:,i])
        y_pred_f = K.flatten(y_pred[:,:,:,i])
        intersection = K.sum(y_true_f * y_pred_f)
        loss = ((2. * intersection + smooth) / (K.sum(y_true_f) + K.
 ↪sum(y_pred_f) + smooth))
    #     K.print_tensor(loss, message='loss value for class {} : '.
 ↪format(SEGMENT_CLASSES[i]))
```

```
        if i == 0:
            total_loss = loss
        else:
            total_loss = total_loss + loss
    total_loss = total_loss / class_num
#    K.print_tensor(total_loss, message=' total dice coef: ')
    return total_loss

# define per class evaluation of dice coef
# inspired by https://github.com/keras-team/keras/issues/9395
def dice_coef_necrotic(y_true, y_pred, epsilon=1e-6):
    intersection = K.sum(K.abs(y_true[:,:,:,1] * y_pred[:,:,:,1]))
    return (2. * intersection) / (K.sum(K.square(y_true[:,:,:,1])) + K.sum(K.
 ↪square(y_pred[:,:,:,1])) + epsilon)

def dice_coef_edema(y_true, y_pred, epsilon=1e-6):
    intersection = K.sum(K.abs(y_true[:,:,:,2] * y_pred[:,:,:,2]))
    return (2. * intersection) / (K.sum(K.square(y_true[:,:,:,2])) + K.sum(K.
 ↪square(y_pred[:,:,:,2])) + epsilon)

def dice_coef_enhancing(y_true, y_pred, epsilon=1e-6):
    intersection = K.sum(K.abs(y_true[:,:,:,3] * y_pred[:,:,:,3]))
    return (2. * intersection) / (K.sum(K.square(y_true[:,:,:,3])) + K.sum(K.
 ↪square(y_pred[:,:,:,3])) + epsilon)

# Computing Precision
def precision(y_true, y_pred):
        true_positives = K.sum(K.round(K.clip(y_true * y_pred, 0, 1)))
        predicted_positives = K.sum(K.round(K.clip(y_pred, 0, 1)))
        precision = true_positives / (predicted_positives + K.epsilon())
        return precision

# Computing Sensitivity
def sensitivity(y_true, y_pred):
    true_positives = K.sum(K.round(K.clip(y_true * y_pred, 0, 1)))
    possible_positives = K.sum(K.round(K.clip(y_true, 0, 1)))
    return true_positives / (possible_positives + K.epsilon())

# Computing Specificity
def specificity(y_true, y_pred):
    true_negatives = K.sum(K.round(K.clip((1-y_true) * (1-y_pred), 0, 1)))
    possible_negatives = K.sum(K.round(K.clip(1-y_true, 0, 1)))
    return true_negatives / (possible_negatives + K.epsilon())
```

[4]:
```
# source https://naomi-fridman.medium.com/
 ↪multi-class-image-segmentation-a5cc671e647a
```

```
def build_unet(inputs, ker_init, dropout):
    conv1 = Conv2D(32, 3, activation = 'relu', padding = 'same',␣
 ↪kernel_initializer = ker_init)(inputs)
    conv1 = Conv2D(32, 3, activation = 'relu', padding = 'same',␣
 ↪kernel_initializer = ker_init)(conv1)

    pool = MaxPooling2D(pool_size=(2, 2))(conv1)
    conv = Conv2D(64, 3, activation = 'relu', padding = 'same',␣
 ↪kernel_initializer = ker_init)(pool)
    conv = Conv2D(64, 3, activation = 'relu', padding = 'same',␣
 ↪kernel_initializer = ker_init)(conv)

    pool1 = MaxPooling2D(pool_size=(2, 2))(conv)
    conv2 = Conv2D(128, 3, activation = 'relu', padding = 'same',␣
 ↪kernel_initializer = ker_init)(pool1)
    conv2 = Conv2D(128, 3, activation = 'relu', padding = 'same',␣
 ↪kernel_initializer = ker_init)(conv2)

    pool2 = MaxPooling2D(pool_size=(2, 2))(conv2)
    conv3 = Conv2D(256, 3, activation = 'relu', padding = 'same',␣
 ↪kernel_initializer = ker_init)(pool2)
    conv3 = Conv2D(256, 3, activation = 'relu', padding = 'same',␣
 ↪kernel_initializer = ker_init)(conv3)


    pool4 = MaxPooling2D(pool_size=(2, 2))(conv3)
    conv5 = Conv2D(512, 3, activation = 'relu', padding = 'same',␣
 ↪kernel_initializer = ker_init)(pool4)
    conv5 = Conv2D(512, 3, activation = 'relu', padding = 'same',␣
 ↪kernel_initializer = ker_init)(conv5)
    drop5 = Dropout(dropout)(conv5)

    up7 = Conv2D(256, 2, activation = 'relu', padding = 'same',␣
 ↪kernel_initializer = ker_init)(UpSampling2D(size = (2,2))(drop5))
    merge7 = concatenate([conv3,up7], axis = 3)
    conv7 = Conv2D(256, 3, activation = 'relu', padding = 'same',␣
 ↪kernel_initializer = ker_init)(merge7)
    conv7 = Conv2D(256, 3, activation = 'relu', padding = 'same',␣
 ↪kernel_initializer = ker_init)(conv7)

    up8 = Conv2D(128, 2, activation = 'relu', padding = 'same',␣
 ↪kernel_initializer = ker_init)(UpSampling2D(size = (2,2))(conv7))
    merge8 = concatenate([conv2,up8], axis = 3)
    conv8 = Conv2D(128, 3, activation = 'relu', padding = 'same',␣
 ↪kernel_initializer = ker_init)(merge8)
```

```
    conv8 = Conv2D(128, 3, activation = 'relu', padding = 'same',␣
 ↪kernel_initializer = ker_init)(conv8)

    up9 = Conv2D(64, 2, activation = 'relu', padding = 'same',␣
 ↪kernel_initializer = ker_init)(UpSampling2D(size = (2,2))(conv8))
    merge9 = concatenate([conv,up9], axis = 3)
    conv9 = Conv2D(64, 3, activation = 'relu', padding = 'same',␣
 ↪kernel_initializer = ker_init)(merge9)
    conv9 = Conv2D(64, 3, activation = 'relu', padding = 'same',␣
 ↪kernel_initializer = ker_init)(conv9)

    up = Conv2D(32, 2, activation = 'relu', padding = 'same',␣
 ↪kernel_initializer = ker_init)(UpSampling2D(size = (2,2))(conv9))
    merge = concatenate([conv1,up], axis = 3)
    conv = Conv2D(32, 3, activation = 'relu', padding = 'same',␣
 ↪kernel_initializer = ker_init)(merge)
    conv = Conv2D(32, 3, activation = 'relu', padding = 'same',␣
 ↪kernel_initializer = ker_init)(conv)

    conv10 = Conv2D(4, (1,1), activation = 'softmax')(conv)

    return Model(inputs = inputs, outputs = conv10)

input_layer = Input((IMG_SIZE, IMG_SIZE, 2))

model = build_unet(input_layer, 'he_normal', 0.2)
model.compile(loss="categorical_crossentropy", optimizer=keras.optimizers.
 ↪Adam(learning_rate=0.001), metrics = ['accuracy',tf.keras.metrics.
 ↪MeanIoU(num_classes=4), dice_coef, precision, sensitivity, specificity,␣
 ↪dice_coef_necrotic, dice_coef_edema ,dice_coef_enhancing] )
```

[5]:
```
# lists of directories with studies
train_and_val_directories = [f.path for f in os.scandir(TRAIN_DATASET_PATH) if␣
 ↪f.is_dir()]

# file BraTS20_Training_355 has ill formatted name for for seg.nii file
train_and_val_directories.remove(TRAIN_DATASET_PATH+'BraTS20_Training_355')

def pathListIntoIds(dirList):
    x = []
    for i in range(0,len(dirList)):
        x.append(dirList[i][dirList[i].rfind('/')+1:])
    return x

train_and_test_ids = pathListIntoIds(train_and_val_directories);
```

```
print(f'total number of pre-labeled MRI scans: {len(train_and_test_ids)}')
# Seperate data into 80% model build, 20% Validation
train_test_ids, val_ids = train_test_split(train_and_test_ids,test_size=0.2)
# 50% of training data to train NN, 50% of training data to calibrate conformal␣
 ↪framework, MUCH less is required for conformal framework to hold
model_ids, conformal_ids = train_test_split(train_test_ids,test_size=0.5)
# Split NN training date into 85% train, 15% test
train_ids, test_ids = train_test_split(model_ids,test_size=0.15)
```

```
total number of pre-labeled MRI scans: 368
```

[6]:
```
class DataGenerator(keras.utils.Sequence):
    'Generates data for Keras'
    def __init__(self, list_IDs, dim=(IMG_SIZE,IMG_SIZE), batch_size = 1,␣
 ↪n_channels = 2, shuffle=True):
        'Initialization'
        self.dim = dim
        self.batch_size = batch_size
        self.list_IDs = list_IDs
        self.n_channels = n_channels
        self.shuffle = shuffle
        self.on_epoch_end()

    def __len__(self):
        'Denotes the number of batches per epoch'
        return int(np.floor(len(self.list_IDs) / self.batch_size))

    def __getitem__(self, index):
        'Generate one batch of data'
        # Generate indexes of the batch
        indexes = self.indexes[index*self.batch_size:(index+1)*self.batch_size]

        # Find list of IDs
        Batch_ids = [self.list_IDs[k] for k in indexes]

        # Generate data
        X, y = self.__data_generation(Batch_ids)

        return X, y

    def on_epoch_end(self):
        'Updates indexes after each epoch'
        self.indexes = np.arange(len(self.list_IDs))
        if self.shuffle == True:
            np.random.shuffle(self.indexes)

    def __data_generation(self, Batch_ids):
```

```
        'Generates data containing batch_size samples' # X : (n_samples, *dim,␣
↪n_channels)
        # Initialization
        X = np.zeros((self.batch_size*VOLUME_SLICES, *self.dim, self.
↪n_channels))
        y = np.zeros((self.batch_size*VOLUME_SLICES, 240, 240))
        Y = np.zeros((self.batch_size*VOLUME_SLICES, *self.dim, 4))


        # Generate data
        for c, i in enumerate(Batch_ids):
            case_path = os.path.join(TRAIN_DATASET_PATH, i)

            data_path = os.path.join(case_path, f'{i}_flair.nii');
            flair = nib.load(data_path).get_fdata()

            data_path = os.path.join(case_path, f'{i}_t1ce.nii');
            ce = nib.load(data_path).get_fdata()

            data_path = os.path.join(case_path, f'{i}_seg.nii');
            seg = nib.load(data_path).get_fdata()

            for j in range(VOLUME_SLICES):
                X[j +VOLUME_SLICES*c,:,:,0] = cv2.resize(flair[:,:
↪,j+VOLUME_START_AT], (IMG_SIZE, IMG_SIZE));
                X[j +VOLUME_SLICES*c,:,:,1] = cv2.resize(ce[:,:
↪,j+VOLUME_START_AT], (IMG_SIZE, IMG_SIZE));

                y[j +VOLUME_SLICES*c] = seg[:,:,j+VOLUME_START_AT];

        # Generate masks
        y[y==4] = 3;
        mask = tf.one_hot(y, 4);
        Y = tf.image.resize(mask, (IMG_SIZE, IMG_SIZE));
        return X/np.max(X), Y

training_generator = DataGenerator(train_ids)
valid_generator = DataGenerator(val_ids)
test_generator = DataGenerator(test_ids)
```

1.2.1 We import pretrained weights to save time, uncomment to self train

[7]:
```
csv_logger = CSVLogger('training.log', separator=',', append=False)


callbacks = [
#     keras.callbacks.EarlyStopping(monitor='loss', min_delta=0,
```

```
#                                patience=2, verbose=1, mode='auto'),
      keras.callbacks.ReduceLROnPlateau(monitor='val_loss', factor=0.2,
                                patience=2, min_lr=0.000001, verbose=1),
#  keras.callbacks.ModelCheckpoint(filepath = 'model_.{epoch:02d}-{val_loss:.
 ↪6f}.m5',
#                                verbose=1, save_best_only=True, save_weights_only␣
 ↪= True)
        csv_logger
    ]
```

[8]:
```
K.clear_session()

# history =  model.fit(training_generator,
#                      epochs=35,
#                      steps_per_epoch=len(train_ids),
#                      callbacks= callbacks,
#                      validation_data = valid_generator
#                      )
# model.save("model_x1_1.h5")
```

[9]:
```
############ load trained model ################
model = keras.models.load_model(os.path.join(os.getcwd(), 'model_per_class.h5'),
                                custom_objects={ 'accuracy' : tf.keras.
 ↪metrics.MeanIoU(num_classes=4),
                                                 "dice_coef": dice_coef,
                                                 "precision": precision,
                                                 "sensitivity":sensitivity,
                                                 "specificity":specificity,
                                                 "dice_coef_necrotic":␣
 ↪dice_coef_necrotic,
                                                 "dice_coef_edema":␣
 ↪dice_coef_edema,
                                                 "dice_coef_enhancing":␣
 ↪dice_coef_enhancing
                                                }, compile=False)

history = pd.read_csv(os.path.join(os.getcwd(), 'training_per_class.log'),␣
 ↪sep=',', engine='python')

hist=history

############### ########## ####### #######

# hist=history.history

acc=hist['accuracy']
```

```
val_acc=hist['val_accuracy']

epoch=range(len(acc))

loss=hist['loss']
val_loss=hist['val_loss']

train_dice=hist['dice_coef']
val_dice=hist['val_dice_coef']

f,ax=plt.subplots(1,4,figsize=(16,8))

ax[0].plot(epoch,acc,'b',label='Training Accuracy')
ax[0].plot(epoch,val_acc,'r',label='Validation Accuracy')
ax[0].legend()

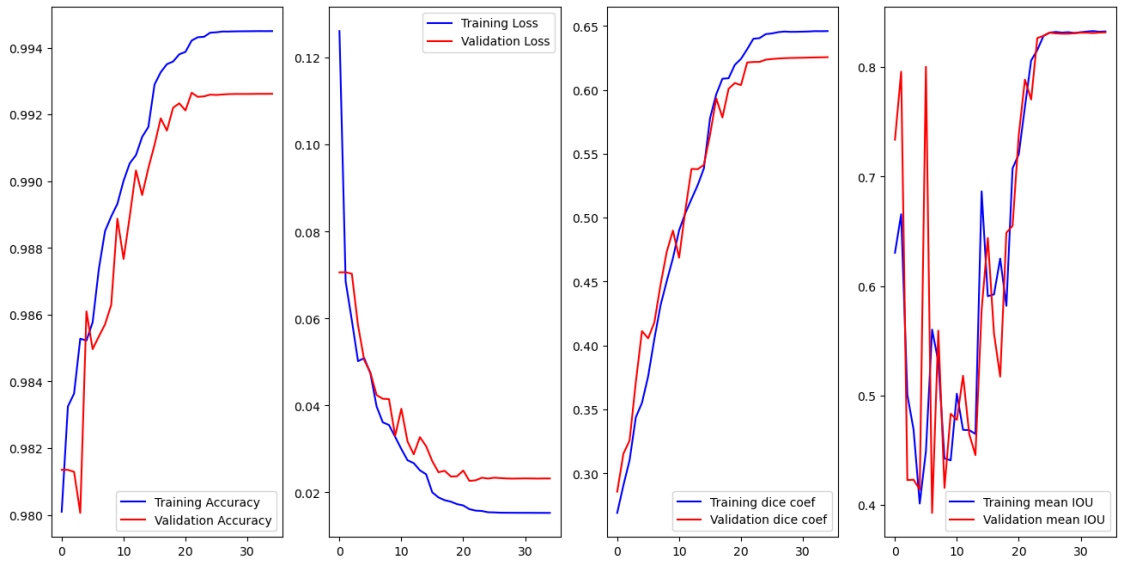
ax[1].plot(epoch,loss,'b',label='Training Loss')
ax[1].plot(epoch,val_loss,'r',label='Validation Loss')
ax[1].legend()

ax[2].plot(epoch,train_dice,'b',label='Training dice coef')
ax[2].plot(epoch,val_dice,'r',label='Validation dice coef')
ax[2].legend()

ax[3].plot(epoch,hist['mean_io_u'],'b',label='Training mean IOU')
ax[3].plot(epoch,hist['val_mean_io_u'],'r',label='Validation mean IOU')
ax[3].legend()

plt.show()
```

```
/opt/anaconda3/lib/python3.11/site-
packages/keras/src/optimizers/base_optimizer.py:34: UserWarning: Argument
`decay` is no longer supported and will be ignored.
  warnings.warn(
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be
built. `model.compile_metrics` will be empty until you train or evaluate the
model.
WARNING:absl:Error in loading the saved optimizer state. As a result, your model
is starting with a freshly initialized optimizer.
```



[10]:
```
# mri type must one of 1) flair 2) t1 3) t1ce 4) t2 ------- or even 5) seg
# returns volume of specified study at `path`
def imageLoader(path):
    image = nib.load(path).get_fdata()
    X = np.zeros((self.batch_size*VOLUME_SLICES, *self.dim, self.n_channels))
    for j in range(VOLUME_SLICES):
        X[j +VOLUME_SLICES*c,:,:,0] = cv2.resize(image[:,:,j+VOLUME_START_AT],␣
 ↪(IMG_SIZE, IMG_SIZE));
        X[j +VOLUME_SLICES*c,:,:,1] = cv2.resize(ce[:,:,j+VOLUME_START_AT],␣
 ↪(IMG_SIZE, IMG_SIZE));

        y[j +VOLUME_SLICES*c] = seg[:,:,j+VOLUME_START_AT];
    return np.array(image)


# load nifti file at `path`
# and load each slice with mask from volume
# choose the mri type & resize to `IMG_SIZE`
def loadDataFromDir(path, list_of_files, mriType, n_images):
    scans = []
    masks = []
    for i in list_of_files[:n_images]:
        fullPath = glob.glob( i + '/*'+ mriType +'*')[0]
        currentScanVolume = imageLoader(fullPath)
        currentMaskVolume = imageLoader( glob.glob( i + '/*seg*')[0] )
        # for each slice in 3D volume, find also it's mask
        for j in range(0, currentScanVolume.shape[2]):
```

```
            scan_img = cv2.resize(currentScanVolume[:,:,j],␣
 ↪dsize=(IMG_SIZE,IMG_SIZE), interpolation=cv2.INTER_AREA).astype('uint8')
            mask_img = cv2.resize(currentMaskVolume[:,:,j],␣
 ↪dsize=(IMG_SIZE,IMG_SIZE), interpolation=cv2.INTER_AREA).astype('uint8')
            scans.append(scan_img[..., np.newaxis])
            masks.append(mask_img[..., np.newaxis])
    return np.array(scans, dtype='float32'), np.array(masks, dtype='float32')

#brains_list_test, masks_list_test = loadDataFromDir(VALIDATION_DATASET_PATH,␣
 ↪test_directories, "flair", 5)
```

2 Visualise Base Tumor Segmentation Classifier

[11]:
```
def predictByPath(case_path, case):
    files = next(os.walk(case_path))[2]
    X = np.empty((VOLUME_SLICES, IMG_SIZE, IMG_SIZE, 2))
  #  y = np.empty((VOLUME_SLICES, IMG_SIZE, IMG_SIZE))

    vol_path = os.path.join(case_path, f'BraTS20_Training_{case}_flair.nii');
    flair=nib.load(vol_path).get_fdata()

    vol_path = os.path.join(case_path, f'BraTS20_Training_{case}_t1ce.nii');
    ce=nib.load(vol_path).get_fdata()

 #   vol_path = os.path.join(case_path, f'BraTS20_Training_{case}_seg.nii');
 #   seg=nib.load(vol_path).get_fdata()


    for j in range(VOLUME_SLICES):
        X[j,:,:,0] = cv2.resize(flair[:,:,j+VOLUME_START_AT],␣
 ↪(IMG_SIZE,IMG_SIZE))
        X[j,:,:,1] = cv2.resize(ce[:,:,j+VOLUME_START_AT], (IMG_SIZE,IMG_SIZE))
 #       y[j,:,:] = cv2.resize(seg[:,:,j+VOLUME_START_AT], (IMG_SIZE,IMG_SIZE))

  #  model.evaluate(x=X,y=y[:,:,:,0], callbacks= callbacks)
    return model.predict(X/np.max(X), verbose=1)

def showPredictsById(case, start_slice = 60):
    path = os.path.join(os.getcwd(), f'BraTS2020_TrainingData/
 ↪MICCAI_BraTS2020_TrainingData/BraTS20_Training_{case[-3:]}')
    gt = nib.load(os.path.join(path, f'BraTS20_Training_{case}_seg.nii')).
 ↪get_fdata()
    origImage = nib.load(os.path.join(path, f'BraTS20_Training_{case}_flair.
 ↪nii')).get_fdata()
    p = predictByPath(path,case)
```

```
    core = p[:,:,:,1]
    edema= p[:,:,:,2]
    enhancing = p[:,:,:,3]

    plt.figure(figsize=(18, 50))
    f, axarr = plt.subplots(1,6, figsize = (18, 50))

    for i in range(6): # for each image, add brain background
        axarr[i].imshow(cv2.resize(origImage[:,:,start_slice+VOLUME_START_AT],␣
↪(IMG_SIZE, IMG_SIZE)), cmap="gray", interpolation='none')

    axarr[0].imshow(cv2.resize(origImage[:,:,start_slice+VOLUME_START_AT],␣
↪(IMG_SIZE, IMG_SIZE)), cmap="gray")
    axarr[0].title.set_text('Original image flair')
    curr_gt=cv2.resize(gt[:,:,start_slice+VOLUME_START_AT], (IMG_SIZE,␣
↪IMG_SIZE), interpolation = cv2.INTER_NEAREST)
    axarr[1].imshow(curr_gt, cmap="Reds", interpolation='none', alpha=0.3) #␣
↪,alpha=0.3,cmap='Reds'
    axarr[1].title.set_text('Ground truth')
    axarr[2].imshow(p[start_slice,:,:,1:4], cmap="Reds", interpolation='none',␣
↪alpha=0.3)
    axarr[2].title.set_text('all classes')
    axarr[3].imshow(edema[start_slice,:,:], cmap="OrRd", interpolation='none',␣
↪alpha=0.3)
    axarr[3].title.set_text(f'{SEGMENT_CLASSES[1]} predicted')
    axarr[4].imshow(core[start_slice,:,], cmap="OrRd", interpolation='none',␣
↪alpha=0.3)
    axarr[4].title.set_text(f'{SEGMENT_CLASSES[2]} predicted')
    axarr[5].imshow(enhancing[start_slice,:,], cmap="OrRd",␣
↪interpolation='none', alpha=0.3)
    axarr[5].title.set_text(f'{SEGMENT_CLASSES[3]} predicted')
    plt.show()


showPredictsById(case=test_ids[0][-3:])
showPredictsById(case=test_ids[1][-3:])
showPredictsById(case=test_ids[2][-3:])
showPredictsById(case=test_ids[3][-3:])
showPredictsById(case=test_ids[4][-3:])
showPredictsById(case=test_ids[5][-3:])
showPredictsById(case=test_ids[6][-3:])

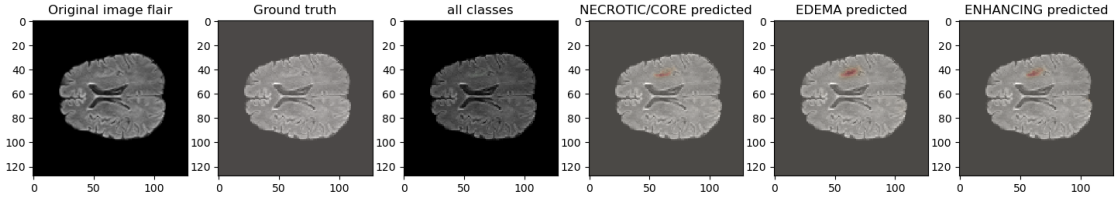

# mask = np.zeros((10,10))
# mask[3:-3, 3:-3] = 1 # white square in black background
# im = mask + np.random.randn(10,10) * 0.01 # random image
```

```
# masked = np.ma.masked_where(mask == 0, mask)

# plt.figure()
# plt.subplot(1,2,1)
# plt.imshow(im, 'gray', interpolation='none')
# plt.subplot(1,2,2)
# plt.imshow(im, 'gray', interpolation='none')
# plt.imshow(masked, 'jet', interpolation='none', alpha=0.7)
# plt.show()
```

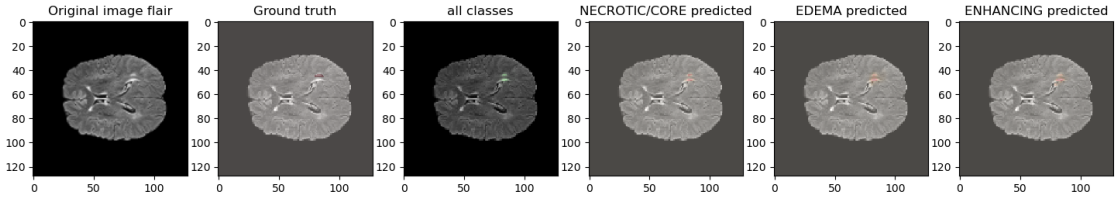
```
4/4                 2s 364ms/step
```

```
<Figure size 1800x5000 with 0 Axes>
```



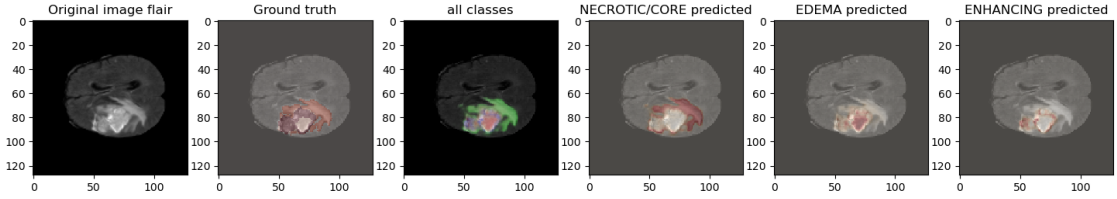

```
4/4                 1s 333ms/step
```

```
<Figure size 1800x5000 with 0 Axes>
```



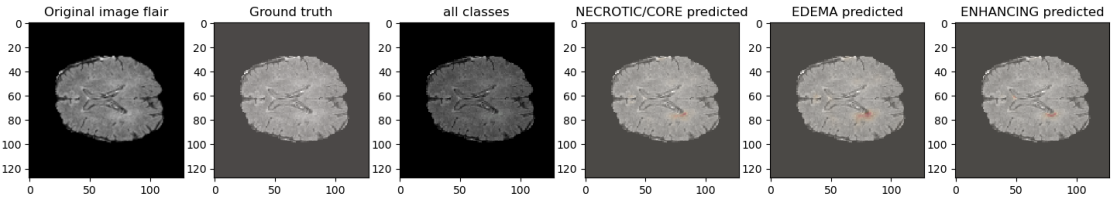

```
4/4                 2s 376ms/step
```

```
<Figure size 1800x5000 with 0 Axes>
```

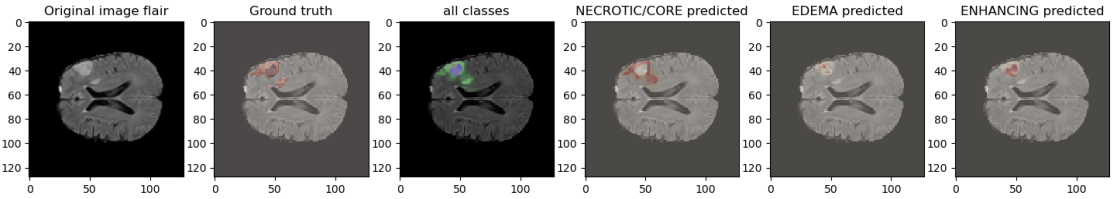



```
4/4                 2s 329ms/step
```

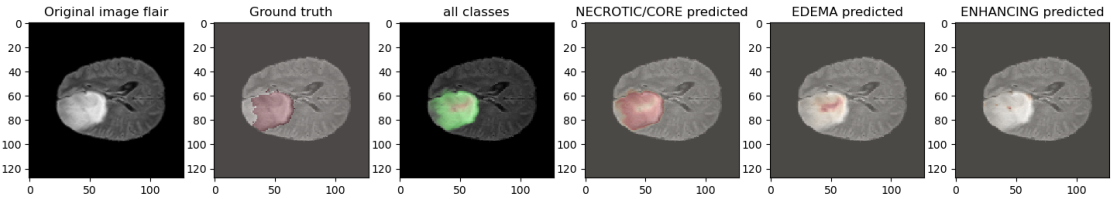
```
<Figure size 1800x5000 with 0 Axes>
```



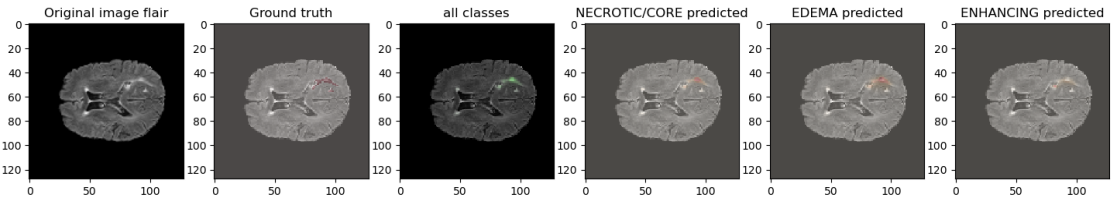

```
4/4                 1s 337ms/step
<Figure size 1800x5000 with 0 Axes>
```




```
4/4                 1s 329ms/step
<Figure size 1800x5000 with 0 Axes>
```




```
4/4                 1s 315ms/step
<Figure size 1800x5000 with 0 Axes>
```



3 Conformal Starts Here

[12]:
```
n = IMG_SIZE * IMG_SIZE * VOLUME_SLICES
target_fnr = 0.01
```

[13]:
```
def conformal_prediction_set(smx_scores, lambda_hat):
    """
    Generates prediction sets based on softmax outputs and a quantile threshold.

    Parameters:
    - smx_scores: A 2D numpy array of softmax outputs. Each row corresponds to␣
 ↪an instance,
                  and each column corresponds to a class.
    - lambda_hat: The quantile threshold used to determine the prediction sets.

    Returns:
    - prediction_sets: A boolean 2D numpy array indicating the classes included␣
 ↪in the
                       prediction set for each instance.
    """
    # Sort the softmax outputs in descending order
    val_pi = smx_scores.argsort(1)[:, ::-1]

    # Calculate the cumulative sums of the sorted softmax outputs
    val_srt = np.take_along_axis(smx_scores, val_pi, axis=1).cumsum(axis=1)

    # Determine the boundary for each instance's prediction set
    exceeds_lambda = val_srt >= lambda_hat
    prediction_set_boundaries = np.argmax(exceeds_lambda, axis=1)

    # # For instances where lambda_hat is never exceeded, include all classes
    # all_included = np.all(~exceeds_lambda, axis=1)
    # prediction_set_boundaries[all_included] = val_srt.shape[1] - 1  # Set to␣
 ↪the last index for these cases

    # Initialize the prediction sets array
    prediction_sets = np.zeros_like(val_srt, dtype=bool)

    # Generate a mask for values within the prediction set boundaries
    col_indices = np.arange(val_srt.shape[1])
    within_boundary_mask = col_indices <= prediction_set_boundaries[:, None]

    # Apply the mask according to the sorted indices
    prediction_sets[np.arange(val_srt.shape[0])[:, None], val_pi] =␣
 ↪within_boundary_mask

    return prediction_sets
```

```
def fnr_loss(prediction_sets, ground_truth):
    """
    Calculates the False Negative Rate (FNR) given prediction sets and ground␣
 ↪truth labels.

    Parameters:
    - prediction_sets: A boolean 2D numpy array indicating the classes included␣
 ↪in the
                       prediction set for each instance.
    - ground_truth: A 1D numpy array of ground truth class labels.

    Returns:
    - FNR: The False Negative Rate as a float.
    """
    # False Negatives: Instances where the ground truth is a tumor (1, 2, or 3),
    # but the prediction set includes only class 0 (no tumor).
    positive_instances = ground_truth > 0
    true_positive_rate = np.any(prediction_sets[positive_instances, 1:],␣
 ↪axis=1).sum() / positive_instances.sum()

    return 1 - true_positive_rate if np.sum(ground_truth > 0) > 0 else 0
```

3.0.1 small example of how conformal prediction sets are formed:

3.0.2 Adaptive Prediction Sets (APS)

[14]:
```
smx_scores = np.array([
    [0.9, 0.05, 0.03, 0.02],  # Slice 1: High confidence for class 0 (No tumor)
    [0.2, 0.5, 0.2, 0.1],     # Slice 2: Moderate confidence for class 1 (Tumor␣
 ↪type 1)
    [0.1, 0.2, 0.35, 0.35],   # Slice 3: Close probabilities for classes 2 and␣
 ↪3 (Tumor types 2 and 3)
    [0.15, 0.15, 0.2, 0.5]    # Slice 4: Leaning towards class 3 (Tumor type␣
 ↪3), but uncertain
])

# Set qhat to define our prediction confidence threshold
lambda_hat = 0.6  # This means we want at least 60% confidence to include a␣
 ↪class in the prediction set

# Deploy
prediction_sets = conformal_prediction_set(smx_scores, lambda_hat)

# Print prediction sets for inspection
print("Prediction Sets:")
print(prediction_sets)
```

```
Prediction Sets:
[[ True False False False]
 [False  True  True False]
 [False False  True  True]
 [False False  True  True]]
```

4 Conformal Calibration Starts here

[15]:
```python
def load_data(case):
    path = os.path.join(os.getcwd(), f'BraTS2020_TrainingData/
 ↪MICCAI_BraTS2020_TrainingData/BraTS20_Training_{case[-3:]}')
    #original_image = nib.load(os.path.join(path, f'BraTS20_Training_{case[-3:
 ↪]}_flair.nii')).get_fdata()
    ground_truth = nib.load(os.path.join(path, f'BraTS20_Training_{case[-3:
 ↪]}_seg.nii')).get_fdata()
    ground_truth[ground_truth == 4] = 3
    ground_truth = cv2.resize(ground_truth, (IMG_SIZE, IMG_SIZE))[:, :,␣
 ↪VOLUME_START_AT:VOLUME_START_AT+VOLUME_SLICES]
    ground_truth = ground_truth.astype(int)
    soft_max = predictByPath(path,case[-3:])
    soft_max = np.transpose(soft_max, (1, 2, 0, 3))
    return soft_max, ground_truth

def process_and_store_data(id_list):
    """
    Process each dataset in id_list and store or return for further processing.
    """
    all_data = []

    for case_id in id_list:
        sftmax, gt = load_data(case_id[-3:])

        all_data.append((sftmax, gt))

    return all_data

conformal_cal_data = process_and_store_data(conformal_ids)
```

```
4/4             1s 320ms/step
4/4             1s 315ms/step
4/4             1s 317ms/step
4/4             1s 327ms/step
4/4             1s 328ms/step
4/4             1s 316ms/step
4/4             1s 338ms/step
4/4             1s 337ms/step
4/4             1s 319ms/step
```

```
4/4             1s 319ms/step
4/4             1s 318ms/step
4/4             1s 346ms/step
4/4             1s 318ms/step
4/4             1s 325ms/step
4/4             1s 317ms/step
4/4             1s 317ms/step
4/4             1s 327ms/step
4/4             1s 319ms/step
4/4             1s 319ms/step
4/4             2s 361ms/step
4/4             1s 323ms/step
4/4             1s 329ms/step
4/4             2s 374ms/step
4/4             1s 332ms/step
4/4             1s 318ms/step
4/4             1s 316ms/step
4/4             1s 316ms/step
4/4             1s 317ms/step
4/4             1s 322ms/step
4/4             1s 322ms/step
4/4             1s 335ms/step
4/4             1s 336ms/step
4/4             1s 317ms/step
4/4             1s 315ms/step
4/4             1s 318ms/step
4/4             1s 316ms/step
4/4             2s 408ms/step
4/4             1s 341ms/step
4/4             1s 322ms/step
4/4             1s 320ms/step
4/4             1s 317ms/step
4/4             1s 318ms/step
4/4             1s 318ms/step
4/4             1s 316ms/step
4/4             1s 320ms/step
4/4             1s 319ms/step
4/4             1s 319ms/step
4/4             1s 316ms/step
4/4             1s 317ms/step
4/4             1s 316ms/step
4/4             1s 316ms/step
4/4             1s 316ms/step
4/4             1s 327ms/step
4/4             1s 344ms/step
4/4             1s 319ms/step
4/4             1s 321ms/step
4/4             1s 317ms/step
```

```
4/4             1s 319ms/step
4/4             1s 318ms/step
4/4             1s 325ms/step
4/4             1s 315ms/step
4/4             1s 321ms/step
4/4             1s 340ms/step
4/4             1s 333ms/step
4/4             1s 321ms/step
4/4             1s 329ms/step
4/4             1s 324ms/step
4/4             1s 326ms/step
4/4             1s 321ms/step
4/4             1s 327ms/step
4/4             1s 323ms/step
4/4             1s 322ms/step
4/4             1s 328ms/step
4/4             1s 324ms/step
4/4             1s 324ms/step
4/4             1s 323ms/step
4/4             1s 323ms/step
4/4             1s 322ms/step
4/4             1s 321ms/step
4/4             1s 319ms/step
4/4             1s 320ms/step
4/4             1s 318ms/step
4/4             1s 320ms/step
4/4             1s 319ms/step
4/4             1s 320ms/step
4/4             1s 320ms/step
4/4             1s 318ms/step
4/4             1s 321ms/step
4/4             1s 325ms/step
4/4             1s 319ms/step
4/4             1s 322ms/step
4/4             1s 320ms/step
4/4             1s 320ms/step
4/4             1s 319ms/step
4/4             1s 321ms/step
4/4             1s 319ms/step
4/4             1s 318ms/step
4/4             1s 339ms/step
4/4             1s 322ms/step
4/4             1s 321ms/step
4/4             1s 322ms/step
4/4             1s 320ms/step
4/4             1s 323ms/step
4/4             1s 324ms/step
4/4             1s 324ms/step
```

```
4/4             1s 320ms/step
4/4             1s 327ms/step
4/4             1s 321ms/step
4/4             1s 322ms/step
4/4             1s 319ms/step
4/4             1s 317ms/step
4/4             1s 316ms/step
4/4             1s 316ms/step
4/4             2s 327ms/step
4/4             1s 322ms/step
4/4             1s 327ms/step
4/4             1s 323ms/step
4/4             1s 325ms/step
4/4             1s 327ms/step
4/4             1s 321ms/step
4/4             1s 321ms/step
4/4             1s 323ms/step
4/4             1s 319ms/step
4/4             1s 329ms/step
4/4             1s 318ms/step
4/4             1s 317ms/step
4/4             1s 317ms/step
4/4             1s 318ms/step
4/4             1s 320ms/step
4/4             1s 316ms/step
4/4             1s 317ms/step
4/4             1s 318ms/step
4/4             1s 316ms/step
4/4             1s 316ms/step
4/4             1s 316ms/step
4/4             1s 317ms/step
4/4             1s 316ms/step
4/4             1s 316ms/step
4/4             1s 317ms/step
4/4             1s 318ms/step
4/4             1s 320ms/step
4/4             2s 447ms/step
4/4             2s 330ms/step
4/4             2s 324ms/step
4/4             1s 324ms/step
4/4             1s 321ms/step
4/4             2s 389ms/step
```

[16]:
```
smx_scores = conformal_cal_data[0][0]
gt_labels = conformal_cal_data[0][1]

smx_scores = smx_scores.reshape(-1, smx_scores.shape[-1])
```

```
gt_labels = gt_labels.flatten()

cal_pi = smx_scores.argsort(1)[:, ::-1]
cal_srt = np.take_along_axis(smx_scores, cal_pi, axis=1).cumsum(axis=1)
cal_scores = np.take_along_axis(cal_srt, cal_pi.argsort(axis=1),axis=1)[np.
 ↪arange(gt_labels.size), gt_labels]
cal_scores[gt_labels>0]
```

[16]:
```
array([0.999, 0.997, 0.997, …, 0.622, 0.992, 0.993], dtype=float32)
```

[17]:
```
# Flatten the softmax scores and ground truth labels across all cases
cal_smx_scores = np.vstack([sftmax.reshape(-1, sftmax.shape[-1]) for sftmax, _␣
 ↪in conformal_cal_data])
cal_gt_labels = np.hstack([gt.flatten() for _, gt in conformal_cal_data])

# Calculate calibration scores for the aggregated data
cal_pi = cal_smx_scores.argsort(1)[:, ::-1]
cal_srt = np.take_along_axis(cal_smx_scores, cal_pi, axis=1).cumsum(axis=1)
cal_scores = np.take_along_axis(cal_srt, cal_pi.argsort(axis=1), axis=1)[np.
 ↪arange(cal_gt_labels.size), cal_gt_labels]
```

[18]:
```
per_example_thresholds = []

for softmax_scores, ground_truth in conformal_cal_data:
    softmax_scores_flat = softmax_scores.reshape(-1, softmax_scores.shape[-1])
    gt_flat = ground_truth.flatten()

    # Get softmax scores for tumor pixels only (exclude background class 0)
    tumor_pixels = softmax_scores_flat[gt_flat > 0, 1:]  # Only classes 1, 2, 3

    if tumor_pixels.shape[0] > 0:
        # Find the per-example quantile threshold
        per_example_lambda = np.quantile(tumor_pixels.min(axis=1), target_fnr)
        per_example_thresholds.append(per_example_lambda)

# Compute final lambda_hat as the global quantile over per-example thresholds
lambda_hat = np.quantile(per_example_thresholds, target_fnr)

print(f"Optimized lambda_hat using per-example quantiles: {lambda_hat}")
```

```
Optimized lambda_hat using per-example quantiles: 1.3709504067982434e-07
```

[19]:
```
def compute_scores(smx_scores, ground_truth):
    """
    Compute per-voxel scores: for each voxel, take the cumulative sum of␣
 ↪softmax scores
```

```
    sorted in descending order and retrieve the score corresponding to the true␣
↪label.

    Parameters:
    - smx_scores: (N, num_classes) array, where N is the number of voxels.
    - ground_truth: (N,) array of true class labels for each voxel.

    Returns:
    - scores: (N,) array of scores for each voxel.
    """
    n = ground_truth.size
    cal_pi = smx_scores.argsort(1)[:, ::-1]  # Sort class indices by descending␣
↪softmax scores
    cal_srt = np.take_along_axis(smx_scores, cal_pi, axis=1).cumsum(axis=1)  #␣
↪Compute cumulative sums

    # Retrieve the cumulative score corresponding to the ground truth class
    scores = np.take_along_axis(cal_srt, cal_pi.argsort(axis=1), axis=1)[np.
↪arange(n), ground_truth]
    return scores[ground_truth > 0]

def compute_lambda_hat_per_example(conformal_cal_data, target_fnr):
    """
    Compute lambda_hat per example (3D scan) instead of pooling over all voxels.

    Parameters:
    - conformal_cal_data: List of tuples (softmax_predictions,␣
↪ground_truth_labels).
    - target_fnr: Target false negative rate for calibration.

    Returns:
    - lambda_hat: The calibrated threshold for prediction sets.
    """
    per_example_lambdas = []

    for smx_scores, gt_labels in conformal_cal_data:
        # Flatten softmax scores and ground truth labels per scan
        smx_scores_flat = smx_scores.reshape(-1, smx_scores.shape[-1])
        gt_labels_flat = gt_labels.flatten()

        # Compute scores per voxel for this example
        scores = compute_scores(smx_scores_flat, gt_labels_flat)

        # Compute the per-example lambda (quantile of scores within this scan)
        per_example_lambda = np.quantile(scores, np.ceil((len(scores) + 1) * (1␣
↪- target_fnr)) / len(scores), interpolation='higher')
```

```
        per_example_lambdas.append(per_example_lambda)

    return per_example_lambdas
```

[20]:
```
lambda_hat = compute_lambda_hat_per_example(conformal_cal_data, target_fnr=0.05)
```

```
/var/folders/4g/c51vpg2j0nj65hpt4ffnh7xm0000gn/T/ipykernel_1637/4035040677.py:43
: DeprecationWarning: the `interpolation=` argument to quantile was renamed to
`method=`, which has additional options.
Users of the modes 'nearest', 'lower', 'higher', or 'midpoint' are encouraged to
review the method they used. (Deprecated NumPy 1.22)
  per_example_lambda = np.quantile(scores, np.ceil((len(scores) + 1) * (1 -
target_fnr)) / len(scores), interpolation='higher')
```

[21]:
```
np.quantile(lambda_hat, np.ceil((len(lambda_hat) + 1) * (0.10)) /␣
↪len(lambda_hat), interpolation='higher')
```

```
/var/folders/4g/c51vpg2j0nj65hpt4ffnh7xm0000gn/T/ipykernel_1637/3241774731.py:1:
DeprecationWarning: the `interpolation=` argument to quantile was renamed to
`method=`, which has additional options.
Users of the modes 'nearest', 'lower', 'higher', or 'midpoint' are encouraged to
review the method they used. (Deprecated NumPy 1.22)
  np.quantile(lambda_hat, np.ceil((len(lambda_hat) + 1) * (0.10)) /
len(lambda_hat), interpolation='higher')
```

[21]:
```
0.9969137
```

[22]:
```
fnr_loss(conformal_prediction_set(cal_smx_scores, 1 - 0.9969137), cal_gt_labels)
```

[22]:
```
0.1440301391035549
```

[23]:
```
# Determine the quantile for lambda_hat that controls the FNR across all data
target_fnr = 0.05  # for example
n = cal_gt_labels.size
# true_positive_scores = cal_scores[cal_gt_labels > 0].min(axis=1)
q_hat = np.quantile(cal_srt[cal_gt_labels > 0, 1:].min(axis=1), np.
↪ceil((n+1)*(1-target_fnr))/n, interpolation='higher')

print(f"Calibrated lambda_hat for all data: {q_hat}")
```

```
Calibrated lambda_hat for all data: 0.9983419179916382

/var/folders/4g/c51vpg2j0nj65hpt4ffnh7xm0000gn/T/ipykernel_1637/2511007291.py:5:
DeprecationWarning: the `interpolation=` argument to quantile was renamed to
`method=`, which has additional options.
Users of the modes 'nearest', 'lower', 'higher', or 'midpoint' are encouraged to
review the method they used. (Deprecated NumPy 1.22)
  q_hat = np.quantile(cal_srt[cal_gt_labels > 0, 1:].min(axis=1),
np.ceil((n+1)*(1-target_fnr))/n, interpolation='higher')
```

[24]:
```
prediction_sets = conformal_prediction_set(cal_smx_scores, 1 - target_fnr)

# Compute the FNR for these prediction sets
current_fnr = fnr_loss(prediction_sets, cal_gt_labels)
```

[25]:
```
current_fnr
```

[25]:
```
0.029468990738599765
```

[26]:
```
# Set your target FNR and range for lambda values
lambda_values = np.linspace(1-1e-5, 0, 1000)  # Starting from 1 (largest set)␣
 ↪to 0 (smallest sets)

# Placeholder for the chosen lambda_hat
lambda_hat = 1  # Start with the largest set (most conservative)

# Loop through lambda values from most conservative (small sets) to least␣
 ↪(large sets)
for lambda_val in lambda_values:
    # Generate prediction sets using the current lambda value
    prediction_sets = conformal_prediction_set(cal_smx_scores, lambda_val)

    # Compute the FNR for these prediction sets
    current_fnr = fnr_loss(prediction_sets, cal_gt_labels)
    print(f'{lambda_val} : {current_fnr}')

    # Check if the current FNR is below the target, if so, update lambda_hat␣
 ↪and continue
    if current_fnr <= target_fnr:
        lambda_hat = lambda_val
    else:
        # If FNR exceeds target, stop the loop; the last lambda_hat is the␣
 ↪largest lambda meeting the FNR criterion
        break

# Print the chosen lambda_hat and the corresponding FNR
print(f"Chosen lambda_hat: {lambda_hat}")
```

```
0.99999 : 0.00027534715038513013
0.998989009009009 : 0.003989237628140407
0.9979880180180181 : 0.00564385595540895
0.996987027027027 : 0.0069234849314624425
0.9959860360360361 : 0.008034001062850127
0.994985045045045 : 0.008989349186838469
0.9939840540540541 : 0.009855957437314156
0.992983063063063 : 0.010674899698588702
0.9919820720720721 : 0.011415243786180529
0.9909810810810811 : 0.012113499819477735
```

0.9899800900900901 : 0.012770935510959092
0.9889790990990991 : 0.01340656654780592
0.9879781081081082 : 0.013999095360375136
0.9869771171171171 : 0.014558156563505342
0.9859761261261262 : 0.015119499649097201
0.9849751351351351 : 0.01565244597516502
0.9839741441441442 : 0.016176771856377337
0.9829731531531531 : 0.01666915138312497
0.9819721621621622 : 0.01716178445236849
0.9809711711711712 : 0.017641233311832982
0.9799701801801802 : 0.01809684917669685
0.9789691891891892 : 0.018567424048809955
0.9779681981981982 : 0.019040280803384935
0.9769672072072072 : 0.01948372662845277
0.9759662162162163 : 0.01990562134138174
0.9749652252252252 : 0.02032219166190008
0.9739642342342343 : 0.020742818662350393
0.9729632432432432 : 0.021147472485568364
0.9719622522522523 : 0.021562014466120605
0.9709612612612613 : 0.021955512419525647
0.9699602702702703 : 0.022370815027565105
0.9689592792792793 : 0.022766848405927576
0.9679582882882883 : 0.02315096528698979
0.9669572972972973 : 0.023537871135505317
0.9659563063063064 : 0.023921227389080202
0.9649553153153153 : 0.024301541132706217
0.9639543243243244 : 0.02465776833923583
0.9629533333333333 : 0.025023123075612497
0.9619523423423424 : 0.025376561314688795
0.9609513513513513 : 0.02573887354111648
0.9599503603603604 : 0.026098903885082403
0.9589493693693694 : 0.026448538986722503
0.9579483783783784 : 0.02680020242832859
0.9569473873873874 : 0.02714299188258351
0.9559463963963964 : 0.02746803336213577
0.9549454054054054 : 0.027808794476424592
0.9539444144144145 : 0.02815107684568796
0.9529434234234234 : 0.028486767110061817
0.9519424324324325 : 0.028822710916931338
0.9509414414414414 : 0.02915307678889445
0.9499404504504505 : 0.029487499340789536
0.9489394594594595 : 0.02981532978779511
0.9479384684684685 : 0.030143413777296346
0.9469374774774775 : 0.03046490566190818
0.9459364864864865 : 0.03079349673640097
0.9449354954954955 : 0.031113974451029813
0.9439345045045046 : 0.03143698759061608
0.9429335135135135 : 0.03176507158011743

0.9419325225225226 : 0.032097719334542196
0.9409315315315315 : 0.03240146324445148
0.9399305405405406 : 0.03270470006936921
0.9389295495495495 : 0.03301757150912554
0.9379285585585586 : 0.033328668151411556
0.9369275675675676 : 0.03363545457126982
0.9359265765765766 : 0.033929056781349054
0.9349255855855856 : 0.034205671644212954
0.9339245945945946 : 0.034517528913986184
0.9329236036036036 : 0.03482659721630621
0.9319226126126127 : 0.03513059466871127
0.9309216216216216 : 0.03541506934894345
0.9299206306306307 : 0.03570613613406515
0.9289196396396396 : 0.03601165484144464
0.9279186486486487 : 0.036305257051523876
0.9269176576576577 : 0.036594295496679585
0.9259166666666667 : 0.03689398272665689
0.9249156756756757 : 0.03718733139424035
0.9239146846846847 : 0.037469270649514996
0.9229136936936937 : 0.037762112232107015
0.9219127027027028 : 0.03805622152717769
0.9209117117117117 : 0.0383452599723334
0.9199107207207208 : 0.03861832524025688
0.9189097297297297 : 0.038885812573273726
0.9179087387387388 : 0.03915786367121421
0.9169077477477477 : 0.03944309897893361
0.9159067567567568 : 0.039721235096772056
0.9149057657657658 : 0.03999151139724222
0.9139047747747748 : 0.04026609792014024
0.9129037837837838 : 0.040543726952987136
0.9119027927927928 : 0.040821102443338364
0.9109018018018018 : 0.04109416771126173
0.9099008108108109 : 0.04136571172421066
0.9088998198198198 : 0.041635988024680826
0.9078988288288289 : 0.04190068639024447
0.9068978378378378 : 0.04215803202343138
0.9058968468468469 : 0.042425772898944
0.9048958558558559 : 0.04269402085944818
0.9038948648648649 : 0.04297672074221015
0.9028938738738739 : 0.043244461617722774
0.9018928828828829 : 0.043500539538430916
0.9008918918918919 : 0.0437725906363714
0.899890900900901 : 0.04404235985185001
0.8988899099099099 : 0.044321256597175784
0.897888918918919 : 0.044578095145371144
0.8968879279279279 : 0.04483062347113875
0.895886936936937 : 0.04508644784935112
0.894885945945946 : 0.04534810370496578

```
0.893884954954955 : 0.045600885573229166
0.892883963963964 : 0.04585113201653501
0.891882972972973 : 0.04612470436944993
0.890881981981982 : 0.04638407834260283
0.889880990990991 : 0.04665080504813246
0.88888 : 0.04691803883865364
0.8878790090090091 : 0.047190850564081344
0.886878018018018 : 0.04745402767467055
0.8858770270270271 : 0.047698189098078325
0.884876036036036 : 0.04796897248354004
0.8838750450450451 : 0.04822378269176941
0.8828740540540541 : 0.04847833935750301
0.8818730630630631 : 0.048731121225766394
0.8808720720720721 : 0.04899125582640651
0.8798710810810811 : 0.0492402345572337
0.8788700900900901 : 0.04949808727541205
0.8778690990990992 : 0.04975289748364142
0.8768681081081081 : 0.049994777024587544
0.8758671171171172 : 0.05024451638290195
Chosen lambda_hat: 0.8768681081081081
```

[ ]:
```python
# Alternatively lambda hat can be found by using binary search which is more␣
 ↪efficient
def binary_search_lambda(all_smx_scores, all_gt_labels, target_fnr, low=0,␣
 ↪high=1, tol=0.0005):
    """
    Use binary search to find the lambda_hat value that achieves the target FNR.
    """
    while low < high - tol:
        print(low, high)
        mid = (low + high) / 2
        prediction_sets = conformal_prediction_set(all_smx_scores, mid)
        current_fnr = fnr_loss(prediction_sets, all_gt_labels)

        if current_fnr > target_fnr:
            low = mid
        else:
            high = mid

    return (low + high) / 2

# Calculate the optimal lambda_hat using binary search
lambda_hat = binary_search_lambda(cal_smx_scores, cal_gt_labels, target_fnr)
print(f"Optimized lambda_hat: {lambda_hat}")
print(fnr_loss(conformal_prediction_set(cal_smx_scores, lambda_hat),␣
 ↪cal_gt_labels))
```

[27]:
```
## Make up space on memory
del conformal_cal_data
del cal_smx_scores
del cal_gt_labels

conformal_val_data = process_and_store_data(val_ids)

# Flatten the softmax scores and ground truth labels across all cases
val_smx_scores = np.vstack([sftmax.reshape(-1, sftmax.shape[-1]) for sftmax, _␣
 ↪in conformal_val_data])
val_gt_labels = np.hstack([gt.flatten() for _, gt in conformal_val_data])
```

```
4/4             2s 336ms/step
4/4             1s 316ms/step
4/4             1s 316ms/step
4/4             1s 321ms/step
4/4             1s 317ms/step
4/4             1s 316ms/step
4/4             1s 318ms/step
4/4             1s 314ms/step
4/4             1s 314ms/step
4/4             1s 315ms/step
4/4             1s 320ms/step
4/4             1s 317ms/step
4/4             1s 313ms/step
4/4             2s 354ms/step
4/4             1s 325ms/step
4/4             1s 332ms/step
4/4             1s 338ms/step
4/4             1s 339ms/step
4/4             1s 317ms/step
4/4             1s 318ms/step
4/4             1s 350ms/step
4/4             1s 335ms/step
4/4             1s 318ms/step
4/4             1s 317ms/step
4/4             1s 314ms/step
4/4             1s 314ms/step
4/4             1s 314ms/step
4/4             1s 320ms/step
4/4             1s 314ms/step
4/4             1s 327ms/step
4/4             1s 330ms/step
4/4             1s 317ms/step
4/4             1s 317ms/step
4/4             1s 316ms/step
4/4             1s 316ms/step
4/4             1s 318ms/step
```

```
4/4             1s 317ms/step
4/4             1s 316ms/step
4/4             1s 316ms/step
4/4             1s 315ms/step
4/4             1s 319ms/step
4/4             1s 322ms/step
4/4             1s 342ms/step
4/4             1s 322ms/step
4/4             1s 309ms/step
4/4             1s 339ms/step
4/4             1s 346ms/step
4/4             1s 322ms/step
4/4             2s 379ms/step
4/4             1s 329ms/step
4/4             1s 335ms/step
4/4             1s 313ms/step
4/4             1s 318ms/step
4/4             1s 340ms/step
4/4             1s 338ms/step
4/4             2s 410ms/step
4/4             1s 318ms/step
4/4             1s 313ms/step
4/4             1s 313ms/step
4/4             1s 315ms/step
4/4             1s 310ms/step
4/4             1s 313ms/step
4/4             1s 313ms/step
4/4             1s 308ms/step
4/4             1s 312ms/step
4/4             1s 314ms/step
4/4             1s 313ms/step
4/4             1s 310ms/step
4/4             1s 314ms/step
4/4             1s 308ms/step
4/4             1s 316ms/step
4/4             1s 320ms/step
4/4             1s 315ms/step
4/4             1s 317ms/step
```

[28]:
```python
# Collect all the FNR for each prediction in the validation set
fnr_list = []
set_size_fraction_list = []

for sftmax, gt in conformal_val_data:
    sftmax_flat = sftmax.reshape(-1, sftmax.shape[-1])
    gt_flat = gt.flatten()
```

```
    prediction_sets = conformal_prediction_set(sftmax_flat, lambda_hat)
    fnr = fnr_loss(prediction_sets, gt_flat)
    fnr_list.append(fnr)

    # Calculate set size as a fraction of polyp size (assuming 'polyp' is␣
 ↪equivalent to tumor)
    # This is an example calculation that you will need to adjust based on your␣
 ↪actual data
    set_size = conformal_prediction_set(sftmax_flat, lambda_hat)[np.
 ↪sum(conformal_prediction_set(sftmax_flat, lambda_hat)[:,1:],axis=1) > 0].
 ↪shape[0]  # Sum over classes
    polyp_size = np.sum(gt_flat > 0)  # Assuming non-zero labels are tumors
    set_size_fraction = set_size / polyp_size
    set_size_fraction_list.extend(set_size_fraction[polyp_size > 0])  # Exclude␣
 ↪cases without polyps

# Generate prediction sets for the validation data using the calibrated␣
 ↪lambda_hat
val_prediction_sets = conformal_prediction_set(val_smx_scores, lambda_hat)

# Calculate the FNR loss on the validation data
validation_fnr_loss = fnr_loss(val_prediction_sets, val_gt_labels)

print(f"Validation FNR: {validation_fnr_loss}")

df = pd.DataFrame({
    'risk': fnr_list,
    'set_size_fraction': set_size_fraction_list
})

# Begin plotting
sns.set(palette='pastel', font='serif')
sns.set_style('white')

plt.rcParams['figure.dpi'] = 300
fig, axs = plt.subplots(nrows=1, ncols=2, figsize=(12,3))

# Plot risk histogram (FNR distribution)
axs[0].hist(df['risk'].to_numpy(), bins=30, alpha=0.7,  density=True)
axs[0].set_xlabel('Risk (False Negative Rate)')
axs[0].locator_params(axis='x', nbins=10)
axs[0].axvline(x=target_fnr,c='#999999',linestyle='--',alpha=0.7)
axs[0].set_ylabel('Density')

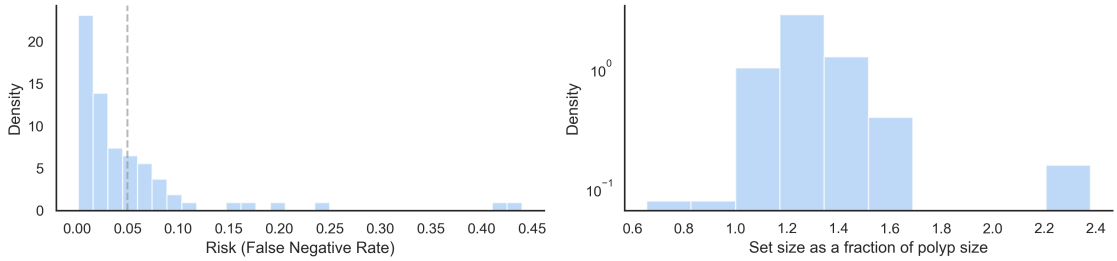
# Plot set size as fraction of polyp size histogram
axs[1].hist(df['set_size_fraction'], alpha=0.7, density=True)
axs[1].set_xlabel('Set size as a fraction of polyp size')
```

```
axs[1].locator_params(axis='x', nbins=10)
axs[1].set_yscale('log')
axs[1].set_ylabel('Density')

sns.despine(top=True, right=True, ax=axs[0])
sns.despine(top=True, right=True, ax=axs[1])
plt.tight_layout()
plt.show()

print(f"The mean and standard deviation of the risk over {len(df)} trials are␣
  ↪{df['risk'].mean():.4f} and {df['risk'].std():.4f} respectively.")
```

```
Validation FNR: 0.04343381762769438
```



The mean and standard deviation of the risk over 74 trials are 0.0521 and 0.0783 respectively.

[29]:
```
# Set the style
sns.set(palette='pastel', font='serif')
sns.set_style('white')

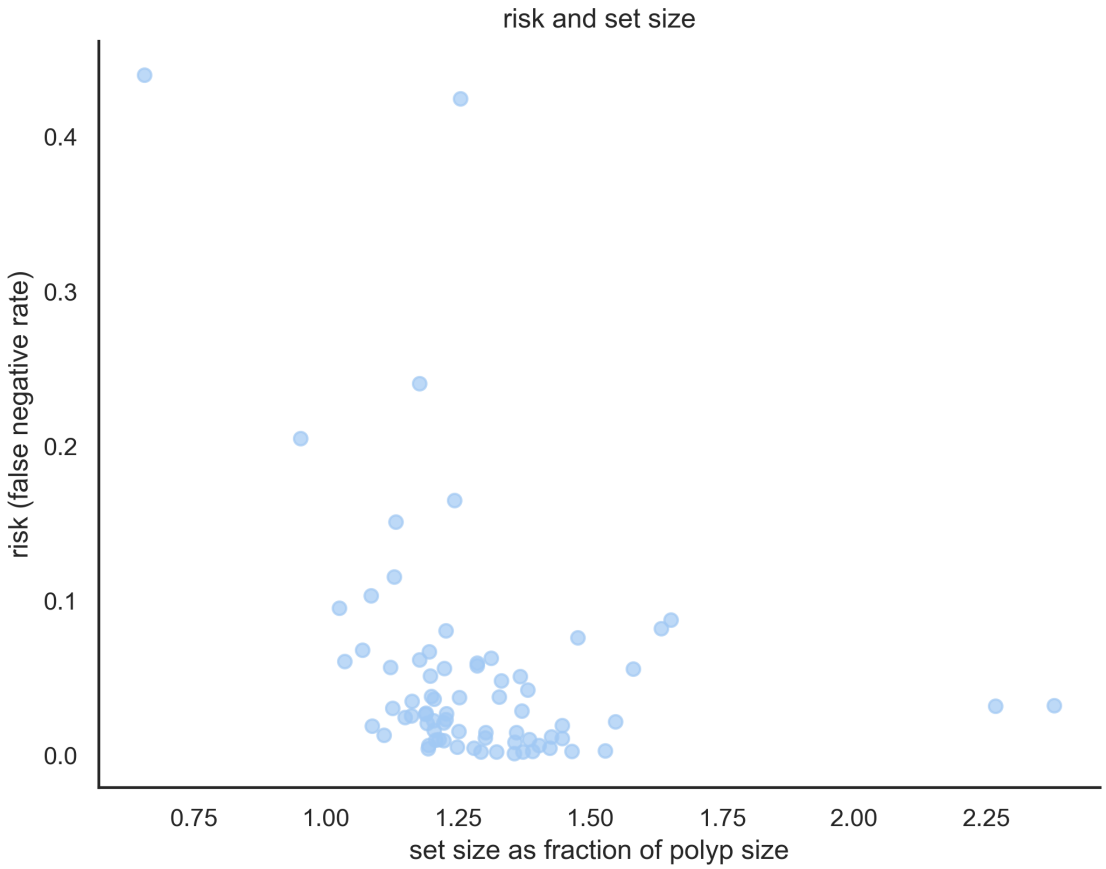
# Begin plotting with the specified figure size and dpi
plt.figure(figsize=(8, 6), dpi=300)
plt.scatter(df['set_size_fraction'], df['risk'], alpha=0.7)

# Setting the scale of x-axis to log scale for set size fractions

# Adding labels and title
plt.xlabel('set size as fraction of polyp size')
plt.ylabel('risk (false negative rate)')
plt.title('risk and set size')

# Remove top and right borders
sns.despine()

# Show the plot
plt.show()
```



[30]:
```
# Generate prediction sets for the validation data using the calibrated␣
↪lambda_hat
val_prediction_sets = conformal_prediction_set(val_smx_scores, lambda_hat)

# Calculate the FNR loss on the validation data
validation_fnr_loss = fnr_loss(val_prediction_sets, val_gt_labels)

print(f"Validation FNR: {validation_fnr_loss}")
```

```
Validation FNR: 0.04343381762769438
```

5 3D Visualisation tool we build for NN Initial Segmentation + Conformal Uncertainty Quantification

Note: there is an error in visual studio code (most likely also in other jupyter notebook IDE's) where files with output larger than 150MB will corrupt the notebook.

hence first clear all outputs running the following part.

Trust me we have found out the hard way ;)

[ ]:
```
# Download Conformal Calibration case
case = '001'
path = os.path.join(os.getcwd(), f'BraTS2020_TrainingData/
 ↪MICCAI_BraTS2020_TrainingData/BraTS20_Training_{case[-3:]}')
cal_gt = nib.load(os.path.join(path, f'BraTS20_Training_{case}_seg.nii')).
 ↪get_fdata()
cal_gt[cal_gt == 4] = 3
cal_gt = cv2.resize(cal_gt, (IMG_SIZE, IMG_SIZE))[:, :, VOLUME_START_AT:
 ↪VOLUME_START_AT+VOLUME_SLICES]
cal_gt = cal_gt.astype(int)
origImage = nib.load(os.path.join(path, f'BraTS20_Training_{case}_flair.nii')).
 ↪get_fdata(); origImage = cv2.resize(origImage, (IMG_SIZE, IMG_SIZE))[:, :,␣
 ↪VOLUME_START_AT:VOLUME_START_AT+VOLUME_SLICES]

cal_smx = predictByPath(path,case)
cal_smx = np.transpose(cal_smx, (1, 2, 0, 3)); p = cal_smx
# Get scores.
cal_smx = cal_smx.reshape(-1, cal_smx.shape[-1])
cal_labels = cal_gt.reshape(-1); n=cal_labels.shape[0]
cal_pi = cal_smx.argsort(1)[:,::-1] # Sorted indices in descending order of␣
 ↪probabilities
cal_srt = np.take_along_axis(cal_smx, cal_pi, axis=1).cumsum(axis=1) #␣
 ↪Cumulative sums of sorted probabilities
cal_scores = np.take_along_axis(cal_srt,cal_pi.
 ↪argsort(axis=1),axis=1)[range(n),cal_labels]
```

[ ]:
```
# Convert probabilities to category assignments
category_assignments = np.argmax(p, axis=-1)

# Ensure there are non-zero categories
if np.all(category_assignments == 0):
    raise ValueError("All category assignments are 0. No tumor categories to␣
 ↪plot.")


# Generate grid coordinates
# np.indices will create a set of arrays (one for each dimension), each␣
 ↪containing the indices along that dimension
x, y, z = np.indices((IMG_SIZE, IMG_SIZE, VOLUME_SLICES))

z = z * 128/244
# Create a figure
fig = go.Figure()

# Create a single volume with multiple isosurfaces
fig.add_trace(go.Volume(
```

```
    x=x.flatten(), # X coordinates
    y=y.flatten(), # Y coordinates
    z=z.flatten(), # Z coordinates
    value=origImage.flatten(), # Assignments
    isomin=1.5,
    isomax=4,  # Adjust based on the number of categories; here it's set to␣
 ↪include 1-3
    opacity=0.1,  # Adjust opacity as needed
    surface_count=1,  # One surface for each category
    colorscale='Greys',  # Use a perceptually uniform colorscale
    name='Brain MRI'
))


# Create a single volume with multiple isosurfaces
fig.add_trace(go.Volume(
    x=x.flatten(), # X coordinates
    y=y.flatten(), # Y coordinates
    z=z.flatten(), # Z coordinates
    value=conformal_prediction_set(cal_smx, lambda_hat).reshape(IMG_SIZE,␣
 ↪IMG_SIZE, VOLUME_SLICES, -1).sum(axis=-1).flatten(), # Assignments
    isomin=1.5,
    isomax=4,  # Adjust based on the number of categories; here it's set to␣
 ↪include 1-3
    opacity=0.5,  # Adjust opacity as needed
    surface_count=3,  # One surface for each category
    colorscale='Reds',  # Use a perceptually uniform colorscale
    name='Conformal Uncertainty'
))

# Create a single volume with multiple isosurfaces
fig.add_trace(go.Volume(
    x=x.flatten(), # X coordinates
    y=y.flatten(), # Y coordinates
    z=z.flatten(), # Z coordinates
    value=category_assignments.flatten(), # Assignments
    isomin=0.5,
    isomax=3.5,  # Adjust based on the number of categories; here it's set to␣
 ↪include 1-3
    opacity=0.2,  # Adjust opacity as needed
    surface_count=3,  # One surface for each category
    colorscale='Jet',  # Use a perceptually uniform colorscale
    name='Tumor Classification'
))


fig.update_layout(
```

```
    title="3D Volumetric Plot of Brain Tumors",
    scene=dict(
        xaxis_title='X Axis',
        yaxis_title='Y Axis',
        zaxis_title='Z Axis'
    ),
    width = 1400,
    height = 900
)

fig.show()
fig.write_html("3D-Tumor-Volume2.html")
```

Gray volume: the brain of the patient

Purple Volume: EDEMA

Green Volume: Necrotic / Core

5.0.1 Conformal Volumes (varying degree of uncertainty)

White Volume: prediction set contains 2 labels

Red Volume: prediction set contains 3 labels

Dark Red Volume: prediction set contains all 4 labels

Since there is only one 'healthy' label, if the prediction set contains more then one label it contains a tumor label.